

Operating Systems - Assignment 2 Report

1. The parts that were added to make the code thread-safe:

```
mutex lockPrint;  
mutex lockTotalNums;  
mutex lockCount;
```

```
if (!threadsArg->TisWorking) {  
    lockPrint.lock();  
    cout << "threadID=" << threadsArg->threadID  
        << ", I have nothing to do!\n";  
    lockPrint.unlock();  
}  
else {  
    lockPrint.lock();  
    cout << "threadID=" << threadsArg->threadID  
        << ", startNum=" << threadsArg->startNum  
        << ", endNum=" << threadsArg->endNum << endl;  
    lockPrint.unlock();  
}
```

Added locks for the print operations inside the thread function to prevent two threads from interrupting each other during the printing of their info.

```
for (int num = threadsArg->startNum; num < threadsArg->endNum; num++) {  
  
    lockTotalNums.lock();  
    totalNums++;  
    lockTotalNums.unlock();  
}
```

totalNums is a global variable that must be locked as well.

```
if (primeCheck) {  
    lockCount.lock();  
    PrimeList.push_back(num);  
    numOfPrime++;  
    lockCount.unlock();  
}  
if (palindromeCheck) {  
    lockCount.lock();  
    PalindromeList.push_back(num);  
    numOfPalindrome++;  
    lockCount.unlock();  
}  
if (primeCheck && palindromeCheck) {  
    lockCount.lock();  
    PalindromicPrimeList.push_back(num);  
    numOfPalindromicPrime++;  
    lockCount.unlock();  
}
```

We also have to lock the part where the global variables (numOfPrime, numOfPalindrome, numOfPalindromicPrime) are being modified, and same thing for the vectors lists since they're global.

Operating Systems - Assignment 2 Report

2. Output of the two versions (thread-safe vs. non-thread-safe):

Over the range of numbers: [0 , 10240000], and using the time command.

- Non-thread-safe:

T = 1	<pre>threadID=0, startNum=0, endNum=10240000 totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m14.414s user 0m13.116s sys 0m1.255s</pre>	
T = 4	Errors due to race condition	<pre>threadID=0, startNum=0, endNum=2560000 threadID=2, startNum=5120000, endNum=7680000 threadID=3, startNum=7680000, endNum=10240000 threadID=1, startNum=2560000, endNum=5120000 double free or corruption (!prev) Aborted real 0m0.009s user 0m0.007s sys 0m0.008s</pre>
T = 16		<pre>double free or corruption (out) Aborted real 0m0.004s user 0m0.006s sys 0m0.000s</pre>
T = 64		<pre>threadID=62, startNum=9920000, endNum=10080000 threadID=63, startNum=10080000, endNum=10240000 Aborted real 0m0.013s user 0m0.021s sys 0m0.000s</pre>
T = 256		<pre>threadID=14, startNum=560000, endNum=600000 Aborted real 0m0.023s user 0m0.034s sys 0m0.005s</pre>
T = 1024		<pre>threadID=4, startNum=40000, endNum=50000 Aborted real 0m0.011s user 0m0.012s sys 0m0.004s</pre>

Operating Systems - Assignment 2 Report

- Thread-safe:

T = 1	<pre>threadID=0, startNum=0, endNum=10240000 totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m14.492s user 0m13.223s sys 0m1.238s</pre>
T = 4	<pre>threadID=1, startNum=2560000, endNum=5120000 threadID=2, startNum=5120000, endNum=7680000 threadID=3, startNum=7680000, endNum=10240000 threadID=0, startNum=0, endNum=2560000 totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m9.175s user 0m15.270s sys 0m1.248s</pre>
T = 16	<pre>totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m8.830s user 0m15.051s sys 0m1.193s</pre>
T = 64	<pre>totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m9.203s user 0m15.523s sys 0m1.381s</pre>
T = 256	<pre>totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m9.119s user 0m14.770s sys 0m1.734s</pre>
T = 1024	<pre>totalNums=10240000, numOfPrime=679461, numOfPalindrome=11023, numOfPalindromicPrime=781 real 0m9.407s user 0m15.516s sys 0m1.780s</pre>

Operating Systems - Assignment 2 Report

3. How many computing units (e.g. cores, hyper-threads) does your machine have?

- **How we arrived at this conclusion from the code:**

Usually, increasing the # of threads will improve the performance and decrease the time needed to finish all tasks, and we noticed that from the output of Task #4, where increasing the # of threads (from 1 to 4 to 16) decreased the time.

But then increasing it more (to 64, 256, 1024) no longer decreased the time and instead it almost stayed the same. That's what "diminishing returns" means.

So, if we assume that hyper-threading (which allows each core to handle 2 threads at the same time) works in our machine, and using (4 - 16 threads) worked the best, then our machine probably has 4 cores (and each core handles 2 threads), so almost 8 threads working at the same time.

If hyper-threading isn't enabled, then about 4 threads are working at the same time.

- **The actual properties of the machine:**

```
[12/20/24] seed@VM:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
Address sizes:          39 bits physical, 48 bits virtual
CPU(s):                2
On-line CPU(s) list:    0,1
Thread(s) per core:     1
Core(s) per socket:     2
Socket(s):              1
```

Socket (Physical CPU) = 1, # of cores in CPU = 2, # of threads per core = 1

Each core handles only one thread at a time, so Hyper-threading isn't actually enabled here.

Only 2 threads work at the same time.